AI 零代码应用生成平台简历写法

编程导航原创项目教程：https://www.codefather.cn

作者：程序员鱼皮

建议

注意，以下简历写法仅供参考，根据自己的简历丰富度、以及对于项目的理解情况有选择地去写。**如果你自己还没有实现项目或者不理解，建议赶紧跟着鱼皮的教程把它弄懂，再写到简历上！**

本项目聚焦于企业级 AI 应用开发的全流程实践，涉及的技术非常前沿且通用。你可以将学到的知识点灵活应用到你的其他项目中，进行整合与升级，突出你的知识广度。比如：

- 使用 LangChain4j 开发 AI 应用
- 使用 LangGraph4j 开发 AI 工作流
- AI 工具调用和幻觉解决
- 基于 AI 的智能路由
- 结合 iframe 与 postMessage 的可视化编辑
- 基于 Selenium 的浏览器自动化
- 基于 Redisson 的分布式限流
- 基于 Nginx 的自动化部署与服务代理
- 基于 ARMS + Prometheus + Grafana 的可观测性
- 利用 Spring Cloud Alibaba 技术栈实现微服务架构

可以使用鱼皮团队开发的 写简历神器《老鱼简历》 快速完整一份简洁专业的简历，有很多现成的简历模板：

专业技能

后端

可以写的内容非常多，大家挑选 4 ~ 8 点即可，千万别全部写上去了！

1. 掌握 Spring Boot 3 和 Java 21 虚拟线程，并能运用 Hutool、Lombok、Knife4j 等类库提高开发效率，遵循阿里 Java 规范保证项目质量。
2. 熟悉 MyBatis Flex 框架，能通过自定义代码生成器和 LambdaQueryWrapper 进行高效的数据访问层开发。
3. 熟悉 LangChain4j 框架，能用 AI Service、结构化输出、护轨、工具调用等特性快速构建企业级 AI 应用。
4. 熟悉 LangChain4j 的对话记忆机制，能够整合 Redis 实现分布式会话的持久化存储。
5. 熟悉 Redis 分布式缓存与 Caffeine 本地缓存，有多级缓存、基于 Redisson 的分布式锁和限流的实践经验。
6. 熟悉 LangGraph4j 工作流引擎，能运用 StateGraph、条件边、并发分支构建高性能的 AI Agent 工作流。
7. 熟悉 Java 并发编程，能使用 CompletableFuture + 自定义线程池加速 I/O 密集型任务。
8. 掌握 Reactor 响应式编程，能构建实时、非阻塞的 AI SSE 流式响应接口。

9. 熟悉 Spring Bean 的 prototype 作用域，能利用其解决有状态 Bean 在并发场景下的问题，如 AI 模型调用的并发瓶颈。
10. 熟悉 LangChain4j 输入护轨，对调用 AI 的请求前置校验，有效防范 Prompt 注入攻击。
11. 熟悉 LangChain4j 输出护轨，对调用 AI 的响应后置校验并配置自动重试策略，增强系统稳定性。
12. 掌握基于 Redisson 的分布式限流技术，能结合 Spring AOP 和自定义注解为核心接口提供灵活的流量保护。
13. 掌握常见设计模式，如门面、策略、模板方法、工厂、执行器模式，并在实际项目中应用优化代码架构。
14. 熟悉 MySQL 数据库设计与索引优化，比如采用游标分页方案解决海量数据的深度分页性能问题。
15. 熟悉 COS 等对象存储服务，不仅能通过 SDK 快速实现文件管理，还能分析优化存储成本。
16. 熟悉 Selenium 浏览器自动化，能结合 WebDriverManager 在无头模式下实现网页截图。
17. 掌握 Prompt 工程和优化技巧，比如思维链、少样本、多视角分析等，能够设计高质量的提示词并减少幻觉。
18. 熟悉第三方 API 的集成与调用，比如图片搜索、AI 文生图等 API 的对接经验。
19. 熟悉 Spring MVC 特性和 Restful API 设计规范，能够通过全局异常处理器全局捕获并处理自定义业务异常。
20. 熟悉 Nginx 作为反向代理和静态资源服务器的核心配置，能解决前后端分离部署中的跨域和路由问题。
21. 熟悉 Linux 服务器常见命令（比如文件操作和进程管理）、运维工具（如 1Panel），能快速完成前后端完整项目的部署。
22. 掌握企业级项目优化技巧，比如利用 ARMS + Prometheus + Grafana 实现全方位的可观测性。
23. 熟悉微服务相关技术栈，如 Spring Cloud Alibaba、Dubbo、Nacos、Higress 网关，能将单体项目快速重构为微服务架构

前端

这个项目的侧重点不在前端，以下仅为示例写法，有部分小细节可能在项目中没有实践。

1. 熟悉 Vue 3 框架及组合式 API，能结合 Vite 和 TypeScript 搭建和开发企业级前端项目。
2. 熟悉 Pinia 全局状态管理库，能设计和实现对用户登录态等全局数据的统一管理及跨组件共享。
3. 熟悉 Vue Router 的路由配置与导航守卫，实现页面懒加载，并利用 beforeEach 守卫进行全局的页面访问权限控制。
4. 熟悉 Ant Design Vue 组件库，能运用其各类组件快速构建管理后台、数据表单和复杂交互界面。

5. 熟悉 SSE 技术，能使用原生 EventSource 消费流式接口，实现 AI 对话的打字机效果。
6. 熟悉 SSE 复杂事件流处理，监听并解析自定义事件，根据不同事件类型更新 UI 状态。
7. 熟悉 iframe 与 postMessage API，能实现主应用与 iframe 沙箱环境之间的安全跨窗口通信。
8. 熟悉动态脚本注入技术，向 iframe 中注入监听脚本，实现对内嵌页面 DOM 元素的可视化选择。
9. 熟悉 markdown-it 和 Highlight.js 等库，实现 Markdown 内容的前端渲染与语法高亮。
10. 熟悉 Axios 请求库，能封装全局请求与响应拦截器，实现 API 错误码的统一处理与多环境配置。
11. 熟悉前端工程化，熟练配置 ESLint 与 Prettier 统一代码规范，并利用 OpenAPI 工具自动生成 API 请求代码。
12. 熟悉前端性能优化，如 CDN 加速、路由懒加载、打包优化、浏览器缓存策略，提升页面加载速度并节省流量。
13. 熟悉前后端联调中的常见问题，比如解决 JavaScript 长整型精度丢失等跨端数据一致性问题。
14. 掌握前端文件下载方案，通过 Fetch API 请求文件流，利用 Blob 和 URL.createObjectURL 触发浏览器下载。
15. 熟悉前端项目部署，使用 Vite 对项目进行打包构建，配合 Nginx 配置 try_files 解决单页应用的路由刷新 404 问题。
16. 掌握前端工具链，包括脚手架、Vite、VS Code、WebStorm、pnpm、Git 等，能够快速搭建与管理前端项目。
17. 能利用 Cursor 等 AI 编程工具高效开发前端界面，通过 Prompt 工程快速生成符合需求的代码。

项目经历

项目名称： XX AI 零代码应用生成平台（比如 “鱼皮 AI 应用生成平台”）

建议根据自己对项目的学习理解程度，自己想个有区分度的名字，其他名称参考：

- XX AI 应用构建器
- XX 智能网站生成器
- XX No Code AI 平台
- XX 网站秒搭
- XX 智能应用生成
- XX AI 项目生成器
- XX 应用魔方
- XX 零代码梦工厂

大家也可以尽情发挥想象力，将其包装为面向特定领域的应用生成平台，比如：

- AI 电商网站生成器
- AI 个人博客搭建助手
- AI 企业官网快速生成
- AI 活动落地页制作工具
- AI 简历作品集生成器

这样一来，就很容易拉开和其他同学简历的区分度。

GitHub 代码地址： https://github.com/liyupi/yu-ai-code-mother

建议大家也把项目放到代码仓库中，并且在主页文档里补充项目架构图、项目功能模块、技术选型等介绍信息。

但是注意不要抄袭鱼皮的写法！一定要加上自己的理解和扩展！ 否则一下就被面试官查出来了。

项目介绍

后端写法

可以根据简历的篇幅对下面的介绍进行适当的删减，一般项目介绍不要超过 3 行

基于 Spring Boot 3 + LangChain4j + LangGraph4j 的 AI 零代码应用生成平台。用户输入自然语言描述，由 AI Agent 自动执行并发素材搜集、代码生成、质量检查、项目构建的完整工作流，最终一键部署为可访问的 Web 应用。

项目核心为一套 AI 工作流，并采用多级缓存、分布式限流、异步处理、护轨重试等多种优化策略，保证系统的高性能与稳定性。

前端写法

基于 Vue 3 + TypeScript + Ant Design Vue 的 AI 零代码应用生成平台。用户能够通过和 AI 多轮对话来生成网站、实时响应式浏览、以及可视化修改网站。

前端基于 SSE 技术实现 AI Agent 执行过程的实时展示，并通过 postMessage 与 iframe 通信，实现了对生成应用的可视化点选编辑功能。（这句话可选）

项目遵循前端工程化标准，通过 ESLint + Prettier 统一代码质量，通过 OpenAPI 自动生成请求代码提高开发效率。

主要工作

根据自己对项目的掌握程度，**选 6 ~ 8 个左右** 去写并适当调整文案，灵活一点。**强烈建议参考鱼皮的项目扩展思路多完善下项目，增加一些区分度！**

后端写法

冒号前的内容只是为了帮助大家快速理解每一点的核心内容，写在简历上时可以移除掉。

1. 项目架构搭建：基于 Spring Boot 3 + Java 21 搭建项目，整合 Hutool、Lombok 和 Knife4j 接口文档，并通过全局异常处理器增强了项目的健壮性。
2. 数据访问层操作：通过自定义 MyBatis Flex 代码生成器快速生成项目基础 CRUD 代码，并结合 LambdaQueryWrapper 实现复杂动态 SQL 查询，减少了大量样板代码。
3. 异常处理：通过自定义业务异常类和错误码规范异常类型，并通过 AOP 实现全局异常处理器，提升了系统容错能力。
4. 权限校验：通过自定义权限注解和 Spring AOP 环绕切面，实现了统一的、非侵入式的用户角色权限校验逻辑。
5. 分布式 Session：整合 spring-session-data-redis，将用户登录态从本地 Session 迁移至 Redis 统一管理，为后续系统的水平扩展打下基础。
6. 应用精选：通过在应用表中设置数字类型的 priority 优先级字段，实现应用的精选功能，并通过添加索引优化查询性能 300%。
7. 【推荐】AI 网站生成服务：利用 LangChain4j 框架集成 DeepSeek、通义等多种 AI 大模型，通过声明式的 Ai Service 编程模型快速实现模型调用和网站生成。
8. 【推荐】Prompt 调优：为代码生成、智能路由、质量检查等不同 AI 任务设计系统提示词，并利用阿里云百炼模型对比能力进行调优，通过少样本等技巧提升 LLM 输出的准确性。
9. 【推荐】设计模式优化：项目实现了原生、Vue 工程等多种网站生成模式，综合运用门面模式统一 AI 调用入口、策略模式与模板方法模式处理各类代码解析和保存逻辑，提升代码的可扩展性。
10. AI 流式响应：基于 LangChain4j 的 TokenStream 和反应式编程，构建 AI 流式响应 SSE 接口；并通过创建同名包和类覆盖源码，为框架增加了工具调用的流式输出能力。
11. 静态网站浏览服务：基于 Spring MVC 开发静态资源访问接口，通过 FileSystemResource 为生成的应用提供隔离的预览服务，并配置目录重定向提升访问地址的兼容性。
12. 【推荐】Nginx 网站部署：通过 Nginx 的 proxy_pass 配置实现 API 请求的反向代理，并通过 alias 和 try_files 为用户生成的多个静态站点提供托管服务，实测单机 QPS 过万。
13. 【推荐】分布式对话记忆：整合 Redis 和 LangChain4j 实现 AI 对话历史的持久化存储，支持通过多轮对话来优化生成的网站，并通过 memoryId 实现了多用户会话的隔离。
14. AI 服务缓存：为解决对话记忆丢失问题，初始化 AI Service 时自动加载数据库中的对话记忆；并通过 Caffeine 缓存同应用的 AI Service 实例，提升服务获取性能并避免重复加载。

15. 【推荐】对话历史查询：基于时间游标实现分页对话历史查询，避免传统的深分页性能问题，并通过建立复合索引进一步优化了查询性能 3 倍。
16. 项目构建：通过 Java 的 Process API 执行网站打包等外部命令，实现了对 Vue 工程项目的自动化构建，并通过超时控制机制防止进程阻塞。
17. 项目打包下载：设计多维文件过滤器，对生成的项目代码进行动态过滤（比如排除隐藏目录）；并通过 Hutool 工具类快速实现 ZIP 压缩，以文件流的形式提供给用户下载。
18. 【推荐】异步优化：将项目构建、封面生成等耗时 I/O 密集型任务异步化，防止请求线程阻塞；通过 Java 21 的虚拟线程执行，避免了传统线程池的资源开销。
19. 【推荐】AI 智能路由：利用 LangChain4j 的结构化输出特性，根据用户需求智能选择不同的网站生成策略，实现成本与性能的平衡。
20. 【推荐】应用封面图生成：基于 Selenium 和 WebDriverManager 实现了自动化网页截图功能，并通过压缩减少了 70% 的空间占用。
21. 【推荐】应用封面图上传：基于 COS 对象存储 SDK 封装了通用文件上传服务，将应用封面图上云持久化存储，并通过定时任务清理过期资源，节约成本。
22. AI 消息处理：将 AI 响应的 SSE 流封装为统一的 JSON 消息载体 ，从而区分 AI 响应、工具调用等多种事件类型，便于前端解析和展示。
23. 【推荐】AI 工具调用：开发了文件读、写、修改、删除、目录读取等文件操作工具，拓宽 AI 能力边界，并通过安全校验防止 AI 误操作导致应用损坏。
24. AI 工具调用的流式输出：通过自定义 TokenStream 流处理器，解析并格式化 AI 实时输出的工具调用信息，将原本黑盒的 Agent 执行步骤透明化，提升用户体验。
25. 【推荐】工具调用优化：通过 @Tool 注解 + Prompt 优化 + AI Service 配置解决工具调用的幻觉问题，实测工具调用触发率 99.9%。
26. 【推荐】集中式工具管理：通过抽象 BaseTool 基类和利用 Spring 自动注入机制实现了 ToolManager，对所有 AI 工具进行统一注册和管理，提升系统的可扩展性。
27. 【推荐】AI 网站生成工作流：基于 LangGraph4j 实现了包含素材搜集、代码生成、质量检查、项目构建的多节点自动化工作流，将复杂的应用生成过程流程化。
28. 【推荐】图片搜集：通过 Pexels API、网页抓取、Mermaid CLI 及阿里云文生图模型等多种工具，赋予 AI 自主搜集素材的能力，提高生成网站的真实性。
29. 【推荐】图片搜集优化：通过 AI 规划图片搜集任务，并通过 CompletableFuture + 自定义线程池并发执行图片搜集，缩短了 300% 的任务总耗时。
30. 【推荐】图片搜集优化：通过 AI 规划图片搜集任务，并通过 LangGraph4j 的 Parallel Branch 并发执行图片搜集，缩短了 300% 的任务总耗时。（和上面那点 2 选 1）
31. 工作流条件边：利用 LangGraph4j 的条件边特性，根据代码生成类型和质量检查结果动态决定执行路径，比如跳过构建或重试生成，进一步提升了流程的自动化程度。
32. AI 代码质量检查：通过访问者设计模式读取生成的项目文件并调用 AI 分析代码，实现对生成代码的自动审查和反馈循环。

33. AI 并发调用优化：通过源码分析排查 AI 只能串行调用的问题，利用 Spring Bean 的多例模式为每次请求生成 ChatModel 实例，优化系统为并行处理。
34. 精选应用缓存：基于 Spring Cache 注解 + Redis 实现首页精选应用列表的缓存，接口响应时长缩短至 xx ms（自己测试）；并通过自定义 CacheManager 设置缓存过期时间和序列化机制。
35. 【推荐】接口限流：基于 Redisson 的 RRateLimiter 和 Spring AOP，通过自定义注解实现了用户 / IP / 接口级的令牌桶限流，保障 AI 接口的稳定性和成本可控。
36. 【推荐】Prompt 安全审查：基于 LangChain4j 的输入护轨对用户 Prompt 进行安全审查，有效拒绝敏感词和防范注入攻击。
37. 【推荐】AI 调用重试：基于 LangChain4j 的输出护轨对 AI 响应进行质量校验并自动触发重试，增强系统稳定性。
38. 优化 SSE 异常处理：改造全局异常处理器，当发生限流等业务异常时，以 SSE 事件流的形式向前端推送结构化的错误信息，提升用户体验。
39. 关联数据清理：在删除应用时，通过重写通用 Service 的 removeById 方法，增加了关联删除其下所有对话历史的逻辑，保证了数据的一致性并节省了空间。
40. 【推荐】AI 成本优化：为代码生成等复杂任务选用推理模型，为路由分类等简单任务选用低成本、快响应的 Turbo 模型，有效降低 AI 成本。
41. 【推荐】项目部署上线：利用 1Panel 面板 + Docker 容器在 Linux 服务器上快速搭建了前后端环境，并通过 Nginx 配置反向代理后端接口，解决了跨域问题。
42. 【推荐】可观测性：基于 ARMS 实现多维系统监控告警、Prometheus + Grafana 实现自定义业务指标监控（如大模型 Token 用量），全方位提升可观测性，便于分析系统。
43. 【推荐】微服务架构：利用 Spring Cloud Alibaba + Dubbo + Nacos + Higress 将单体项目重构为微服务架构，实现用户服务、AI 服务、截图服务的弹性扩展和故障隔离，提高系统可维护性和容错性。

前端写法

1. 基于 Vue 3 + Vite + TypeScript 搭建前端项目，并利用 Pinia 实现对用户登录态的全局状态管理。
2. 响应式布局：利用 Ant Design Vue 的栅格系统和媒体查询，实现了多端兼容的 AI 聊天和网站实时浏览页面，保证不同设备下的用户体验。
3. AI 实时流式响应：通过原生 EventSource 消费后端 SSE 流式接口，实现 AI 对话的打字机效果及工具调用过程的实时展示，提升用户体验。
4. 可视化编辑网站：利用 postMessage API 实现了主应用与 iframe 中预览站点的跨窗口通信，并动态注入监听脚本，实现用户通过点击元素来可视化编辑网站。
5. 集中权限管理：基于 Vue Router 的 meta 属性与 beforeEach 全局路由守卫，实现了集中式的页面访问权限控制方案，能根据用户角色动态渲染导航菜单。
6. 前端工程化：配置 ESLint + Prettier 统一代码规范，并利用 OpenAPI 工具根据后端 Swagger 文档自动生成 TypeScript 模型与 API 请求代码。
7. 组件封装：基于 Ant Design Vue 封装了应用卡片、详情弹窗、部署成功弹窗等多个业务组件，通过 props 和 emits 进行父子组件通信，提高了代码复用率。

8. SSE 事件：设计实现了 SSE 自定义事件的监听和处理逻辑，能够针对限流、业务异常等情况给出友好的错误提示，提升了用户体验。
9. 文件下载：基于 Fetch API 请求文件流，并利用 Blob 和 URL.createObjectURL 创建临时链接实现代码压缩包的下载。并利用 Content-Disposition 响应头解析下载文件名。
10. AI Vibe Coding：在开发中使用 Cursor 等 AI 编程工具，通过编写高质量 Prompt 快速生成页面基础结构和组件代码。
11. 项目部署上线：使用 Vite 对项目进行打包构建，配合 Nginx 完成静态资源部署，并通过配置 try_files 解决了单页应用刷新 404 的问题。
12. 解决前端数据精度问题：利用全局 Axios 响应拦截器和开发者工具进行调试，通过后端配置，有效解决了因 JavaScript Number 类型限制导致的长整型 ID 精度丢失问题。

个人评价

1. 具备扎实的 AI 应用开发能力，熟练掌握 LangChain4j 及 LangGraph4j 框架，能独立构建包含工具调用、多轮对话记忆和可编排工作流的复杂 AI Agent。
2. 具备企业级 AI 系统架构设计能力，能从性能（并发优化、多级缓存）、安全（接口限流、注入防范）、成本（模型选型）、稳定性（重试机制）等多维度进行优化。
3. 自主学习和问题解决能力较强，能通过阅读官方文档、GitHub Issues 自学 LangGraph4j 等前沿技术，并通过覆盖依赖源码等方式解决框架局限性问题。
4. 具备优秀的产品思维，懂得调研大厂项目来设计需求，比如为增强用户体验，实现了 AI 工具调用的 SSE 流式输出等。
5. 善于将复杂 AI 技术与实际业务场景结合，通过工具调用、MCP 服务集成等方式扩展 AI 能力边界，解决实际问题。
6. 熟练掌握 Prompt 工程技巧，能通过思维链、少样本学习等优化手段显著提升大模型输出质量，实现更精准的 AI 响应。
7. 注重工程实践与代码质量，能熟练运用门面、策略、模板方法等多种设计模式优化项目架构；并深度使用 AI 编程工具提升开发效率。